CU-Connect — Going Live

Step-by-step production deployment: **MongoDB Atlas** (database) → **Render** (Express + Socket.IO backend) → **Vercel** (static frontend).

Phase 0 — Get the code on GitHub

Render and Vercel deploy from git. From `D:\Vibe Coding\2\CU - Connect`:

```
cd "D:/Vibe Coding/2/CU - Connect"
git init
git add .
git commit -m "CU-Connect production build"
```

Create an empty repo on github.com (e.g. `cu-connect`), then:

```
git remote add origin https://github.com/<you>/cu-connect.git
git branch -M main
git push -u origin main
```

> **Important:** push the *CU - Connect* folder itself as the repo root, so the repo contains Backend/ and Frontend/ at the top level. .gitignore already excludes node_modules and .env — never commit those.

Phase 1 — MongoDB Atlas (database)

- Sign up at mongodb.com/cloud/atlas → **Create** a free **M0** cluster (region near your users).
- **Database Access** → Add New Database User → username + strong password. Role: Read and write to any database.
- **Network Access** → Add IP Address → **Allow access from anywhere** (0.0.0.0/0). Render IPs are dynamic, so this is required on the free tier.
- **Database** → **Connect** → **Drivers** → copy the connection string, insert your password, and add the DB name `cu-connect` before the '?':

```
mongodb+srv://myuser:mypass@cluster0.xxxxx.mongodb.net/cu-connect?retryWrites=true&w=majority
```

Keep this — it is your **MONGODB_URI**.

Phase 2 — Render (backend: Express + Socket.IO)

- Sign up at render.com, connect GitHub.
- **New** → **Web Service** → pick your repo.

Settings:

- **Root Directory:** `Backend`
- **Runtime:** Node **Build:** `npm install` **Start:** `npm start`
- **Instance Type:** Free **Health Check Path:** `/healthz`

Environment Variables:

Key	Value
`NODE_ENV`	`production`
`JWT_SECRET`	`node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`
`MONGODB_URI`	your Atlas string from Phase 1

`IDE_ENABLED`	`false`
`CORS_ORIGINS`	leave blank for now — filled in Phase 4

- **Create Web Service.** Wait for green. Copy the URL, e.g. https://cu-connect-backend.onrender.com.
- Test: open `/healthz` → returns {"ok":true,"ide":false}. The DB auto-seeds on first boot.

The Backend/render.yaml blueprint automates this via 'New → Blueprint', but its rootDir assumes a parent folder. The manual steps above are the reliable path.

Phase 3 — Vercel (frontend: static HTML)

- Sign up at vercel.com, connect GitHub.
- **Add New** → **Project** → pick your repo.
- **Root Directory:** `Frontend/Landing Page` (click Edit and select it).
- **Framework Preset:** Other. Leave build/output empty (static HTML).
- **Deploy.** Copy the URL, e.g. https://cu-connect.vercel.app.

The site loads but login will fail — the frontend is still same-origin. Fixed in Phase 4.

Phase 4 — Wire the two origins together

a) Point the frontend at the backend. Edit `Frontend/Landing Page/config.js`:

```
window.CUCONNECT = {
  API_BASE: 'https://cu-connect-backend.onrender.com',
  SOCKET_URL: 'https://cu-connect-backend.onrender.com',
};
```

Commit and push — Vercel auto-redeploys:

```
git commit -am "Point frontend at Render backend"
git push
```

b) Allow the frontend origin on the backend. In Render → Environment, set:

```
CORS_ORIGINS = https://cu-connect.vercel.app
```

Add custom domains too, comma-separated, no trailing slash. Save → Render redeploys automatically.

Phase 5 — Verify live

- Open your Vercel URL → login.html.
- Log in with a seeded demo account: pallakshi@chitkara.edu.in / password123.
- Check the dashboard loads (attendance %, CGPA). Open a teacher session (rajesh@chitkara.edu.in) in another browser, mark attendance, and confirm the student view updates live — that proves Socket.IO + CORS work across origins.

Free-tier notes

- **Render spins down after ~15 min idle.** First request after that takes ~30–50s (cold start); Socket.IO reconnects once up. Upgrade the instance or add an uptime pinger to stay hot.
- **IDE stays disabled** (IDE_ENABLED=false). /api/execute returns 503 by design — the local runner is unsandboxed. Wire lib/runner.js to a hosted Judge0/Docker sandbox before enabling.
- **Re-seeding:** the DB seeds only when empty. To reset demo data, drop the cu-connect database in Atlas and restart the Render service.

• **Never commit .env** — it is gitignored. All secrets live in the Render dashboard.